

ЛАБОРАТОРНА РОБОТА № 5

5.1 Тема

Робота з текстовими файлами: читання та запис

5.2 Мета роботи

Ознайомитися з основами роботи з файлами у Python: відкрити файл для читання та запису, зчитувати та модифікувати його вміст, здійснювати обробку рядків і запис результатів у новий файл.

5.3 Навчальні результати

Після виконання лабораторної роботи здобувач освіти повинен:

1) знати:

- основні режими відкриття файлів у Python (**r**, **w**, **a**, **r+**);
- методи роботи з файлами: **read()**, **readline()**, **readlines()**, **write()**, **writelines()**;
- принципи збереження та закриття файлів (**close()**) та конструкція **with**;
- основи обробки рядків при роботі з файлами (зміна регістру, пошук та заміна тексту).

2) уміти:

- відкривати текстові файли для читання та запису;
- зчитувати дані з файлів і записувати їх у нові файли;
- виконувати перетворення та обробку тексту з файлу (підрахунок слів, сортування, зміна регістру);
- додавати нові дані до існуючого файлу та перезаписувати файли за потреби;
- перевіряти наявність файлу перед роботою з ним.

3) набути навичок:

- розробки простих програм для обробки текстових файлів;
- роботи з потоками даних у Python (читання/запис);
- використання конструкції **with** для безпечного управління файлами;

– самостійного аналізу вмісту файлу та виконання необхідних перетворень даних.

5.4 Матеріально-технічне забезпечення

- персональний комп'ютер;
- операційна система Windows / Linux / macOS;
- середовище програмування (IDLE, PyCharm, VS Code або онлайн-інтерпретатор Python);
- встановлений інтерпретатор Python версії 3.x.

5.5 Теоретичні відомості

1. Основи роботи з файлами у Python

У Python файл – це зовнішній ресурс для збереження інформації, який дозволяє зчитувати дані та записувати їх у постійне сховище. Файли бувають:

- текстові (.txt, .csv) – містять символи у звичайному форматі;
- бінарні (.bin, .jpg) – містять дані у двійковому форматі.

Для роботи з файлами використовуються вбудовані функції Python, головна з яких – ***open()***.

Відкриття файлу

Функція ***open()*** має наступний синтаксис:

```
file = open('назва_файлу', 'режим', encoding='utf-8')
```

Аргументи:

- 'назва_файлу' – шлях до файлу;
- 'режим' – режим роботи з файлом (див. нижче);
- encoding='utf-8' – кодування тексту (рекомендовано завжди вказувати для уникнення помилок з кирилицею).

Основні режими відкриття файлу

Режим	Опис
'r'	Читання (файл повинен існувати)
'w'	Запис з перезаписом (якщо файл існує – він буде стертий)
'a'	Дозапис (нові дані додаються в кінець файлу)
'r+'	Читання та запис одночасно

Закриття файлу

Після завершення роботи файл потрібно закрити, щоб звільнити ресурс та зберегти зміни:

```
file.close()
```

Альтернатива – конструкція **with**, яка автоматично закриває файл після завершення блоку коду:

```
with open('file.txt', 'r', encoding='utf-8') as f:
    data = f.read()
# Файл автоматично закритий після виходу з блоку
```

Зчитування даних з файлів**Метод read()**

Зчитує весь вміст файлу як один рядок:

```
with open('file.txt', 'r', encoding='utf-8') as f:
    text = f.read()
```

Метод readline()

Зчитує один рядок за раз:

```
with open('file.txt', 'r', encoding='utf-8') as f:
    line = f.readline()
```

Можна викликати **readline()** кілька разів, щоб пройтися по всіх рядках файлу.

Метод readlines()

Зчитує всі рядки у список:

```
with open('file.txt', 'r', encoding='utf-8') as f:
    lines = f.readlines() # lines – список рядків
```

Запис даних у файли

Метод write()

Записує рядок у файл. Якщо файл відкритий у режимі 'w', попередній вміст перезаписується:

```
with open('output.txt', 'w', encoding='utf-8') as f:
    f.write("Привіт, Python!\n")
```

Метод writelines()

Записує список рядків у файл:

```
lines = ['Перший рядок\n', 'Другий рядок\n']
with open('output.txt', 'w', encoding='utf-8') as f:
    f.writelines(lines)
```

Важливо: **writelines()** не додає автоматично символ кінця рядка, його треба вставляти вручну (**\n**).

Обробка рядків при роботі з файлами

Під час роботи з текстовими файлами часто потрібно виконувати обробку даних:

Зміна регістру:

```
line = line.upper() # у верхній регістр
line = line.lower() # у нижній регістр
```

Пошук підрядків:

```
if 'Python' in line:
    print("Знайдено!")
```

Підрахунок слів:

```
word_count = len(line.split())
```

Сортування даних:

```
numbers = [int(x) for x in line.split()]
numbers.sort()
```

Видалення порожніх рядків:

```
lines = [l for l in lines if l.strip() != '']
```

Практичні поради

- 1) Завжди перевіряйте наявність файлу перед читанням:

```
import os
if os.path.exists('file.txt'):
    with open('file.txt', 'r', encoding='utf-8') as f:
        data = f.read()
```

- 2) Використовуйте контекстний менеджер **with** для безпечного закриття файлу.
- 3) Для роботи з кількома файлами одночасно можна використовувати кілька блоків **with**.
- 4) При роботі з великими файлами краще зчитувати їх построчно, щоб не перевантажувати пам'ять.

Приклад інтегрованого використання

```
# Зчитування даних, обробка та запис у новий файл
with open('input.txt', 'r', encoding='utf-8') as infile:
    lines = infile.readlines()

# Перетворення: нижній регістр, видалення порожніх рядків
processed_lines = [line.lower() for line in lines if line.strip() != '']

# Запис результату
with open('output.txt', 'w', encoding='utf-8') as outfile:
    outfile.writelines(processed_lines)

print("Обробка завершена. Результат збережено в output.txt")
```

Цей приклад демонструє повний цикл:

- відкриття файлу для читання,
- обробку рядків,
- запис у новий файл,
- безпечне закриття файлів.

5.6 Порядок виконання роботи

- 1) Запустити середовище програмування Python.
- 2) Створити новий файл програми.
- 3) Ознайомитися з теоретичними відомостями до роботи.

- 4) Реалізувати загальне завдання лабораторної роботи.
- 5) Виконати індивідуальне завдання відповідно до варіанта.
- 6) Перевірити коректність роботи програми.
- 7) Оформити звіт та подати його на перевірку.

5.7 Загальне завдання

Написати програму, яка зчитує дані з текстового файлу та записує їх у новий файл, виконуючи певні перетворення (наприклад, переведення всіх літер у нижній регістр).

5.8 Індивідуальні завдання

- 1) Зчитати дані з файлу та записати їх у новий файл.
- 2) Підрахувати кількість слів у файлі та записати результат у новий файл.
- 3) Зчитати числа з файлу та обчислити їх середнє значення.
- 4) Записати дані у файл у зворотному порядку.
- 5) Перевірити наявність певного слова у файлі.
- 6) Замінити певне слово у файлі на інше.
- 7) Додати новий рядок у існуючий файл.
- 8) Записати список чисел у файл та відсортувати їх.
- 9) Видалити всі порожні рядки з файлу.
- 10) Перезаписати файл лише якщо він існує.

Нульовий варіант (приклад реалізації)

Написати програму, яка:

- 1) Зчитує дані з текстового файлу input.txt.
- 2) Перетворює весь текст у нижній регістр.
- 3) Видаляє порожні рядки.
- 4) Записує результат у новий файл output.txt.

Цей варіант можна вважати базовим, який демонструє весь процес роботи з файлами і підготовлює студентів до складніших індивідуальних завдань.

Програмний код

```
# Лабораторна робота №5
# Нульовий варіант
# Робота з текстовими файлами: читання та запис

# Імпортуємо модуль для перевірки існування файлу
import os

# Перевірка наявності вхідного файлу
if not os.path.exists('input.txt'):
    print("Файл input.txt не знайдено. Створіть файл та додайте дані.")
else:
    # Відкриваємо файл для читання у кодуванні UTF-8
    with open('input.txt', 'r', encoding='utf-8') as infile:
        lines = infile.readlines() # Зчитуємо всі рядки у список

    # Обробка рядків: переводимо в нижній регістр та видаляємо порожні рядки
    processed_lines = [line.lower() for line in lines if line.strip() != '']

    # Відкриваємо новий файл для запису результату
    with open('output.txt', 'w', encoding='utf-8') as outfile:
        outfile.writelines(processed_lines) # Записуємо оброблені рядки

    print("Обробка файлу завершена. Результат збережено у 'output.txt'.")
```

Пояснення коду:

1) `import os` — підключаємо модуль для роботи з файлами (перевірка наявності).

2) `os.path.exists('input.txt')` — перевіряє, чи існує вхідний файл.

3) `with open('input.txt', 'r', encoding='utf-8') as infile:` — відкриваємо файл для читання; `with` автоматично закриває файл після завершення блоку.

4) `lines = infile.readlines()` — зчитуємо всі рядки з файлу у список.

5) `processed_lines = [line.lower() for line in lines if line.strip() != '']` — обробляємо рядки: переводимо у нижній регістр та видаляємо порожні рядки.

6) `with open('output.txt', 'w', encoding='utf-8') as outfile:` — відкриваємо новий файл для запису результату.

7) `outfile.writelines(processed_lines)` — записуємо список рядків у файл.

8) `print(...)` — повідомляємо користувача про завершення роботи.

5.9 Вимоги до звіту

Звіт з лабораторної роботи повинен містити:

- 1) Назву лабораторної роботи.
- 2) Мету роботи.
- 3) Короткі теоретичні відомості.
- 4) Умову загального та індивідуального завдання.
- 5) Текст програми з коментарями.
- 6) Результати виконання програми (скріншоти або приклади виводу).
- 7) Висновок.

5.10 Контрольні питання

- 1) Як відкрити файл для читання в Python?
- 2) Які методи використовуються для зчитування файлів (read(), readline(), readlines())?
- 3) Як додавати дані у файл без перезапису?
- 4) Як перевірити наявність файлу перед його відкриттям?
- 5) Як правильно закривати файл після роботи з ним?

5.11 Літературні джерела

- 1) Python Software Foundation. – URL: <http://www.python.org>
- 2) Яковенко А. В. Основи програмування. Python. Частина 1 [Електронний ресурс]: підручник для студентів спеціальності 122 «Комп'ютерні науки». Київ: КПІ ім. Ігоря Сікорського, 2018. 195 с.
<https://ela.kpi.ua/bitstream/123456789/25111/1/Python.pdf>